

Project Interim Report On

Smart Parking Management Project

(Using Python Scripts, ML MODELS)

RU-100-01-00002
Foundation of Machine Learning

SESSION: 2025-26



Submitted By
PIYUSH KUMAR
KARSH
RU-25-10967

Master of Computer Application in
Association with Google

Under the Guidance of
Mr. Zamin Hamid

SCHOOL OF INFORMATION AND TECHNOLOGY

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

ABSTRACT

The Smart Parking Management System is a web-based application developed to solve the common problem of inefficient parking management. Finding an available parking slot wastes time, causes traffic congestion, and increases fuel consumption. This project provides a simple and functional solution by combining web development with machine learning.

The system displays real-time parking slot availability and allows administrators to manage slot occupancy manually or through simulated sensor data. Every slot change is automatically saved to a database to build historical records. A Random Forest machine learning model analyses this historical data based on time of day and day of week to predict whether parking slots will be available in the next hour. The system is built using Python Flask for the backend, SQLite for data storage, Bootstrap for the frontend interface, and Scikit-learn for machine learning. The application opens directly to the admin dashboard without requiring any login, making it simple and straightforward to use for demonstration and academic purposes.

1. INTRODUCTION

1.1 Background

The number of vehicles in urban areas has increased significantly over the years. One of the biggest daily challenges for vehicle owners is finding an available parking space. Drivers waste time driving around parking lots looking for a free slot, which leads to traffic congestion, wasted fuel, and unnecessary stress. Traditional parking systems are entirely manual and do not provide any way to know in advance whether a parking space will be available.

1.2 Problem Statement

Existing parking systems have no real-time monitoring and no predictive capability. There is a need for a smarter system that can track parking slot occupancy, store historical usage data, and use that data to predict future availability.

1.3 Proposed Solution

The Smart Parking Management System addresses this problem by providing the following:

- **Real-time slot monitoring** showing which slots are occupied and which are free
- **Admin dashboard** to manage and update all slot statuses from one place
- **Sensor simulation** to mimic real IoT sensor behaviour without physical hardware
- **Historical data logging** that automatically records every slot change with time and date
- **Machine learning prediction** that forecasts parking availability for the next hour
- **Occupancy trend chart** showing average parking usage patterns by hour of day

1.4 Key Features

- Direct access dashboard with no login required
- Color-coded parking slot cards (green for available, red for occupied)
- One-click sensor simulation for all ten parking slots
- ML model retraining directly from the admin panel
- REST API endpoints for live slot and prediction data
- Auto-refreshing parking view every thirty seconds

2. CORE OBJECTIVES

The main objectives of this project are as follows:

- **Manage Parking Slot Records** — Maintain a real-time database of all ten parking slots and their current occupancy status
- **Track Slot History** — Automatically record every slot status change with timestamp, day of week, and hour of day into a history table
- **Reduce Manual Effort** — Allow administrators to simulate sensor data for all slots at once instead of updating each slot individually
- **Apply Machine Learning** — Use a Random Forest Classifier to analyze historical patterns and predict next hour parking availability
- **Display Visual Analytics** — Show an occupancy trend bar chart built from historical data using Chart.js
- **Provide API Access** — Expose JSON API endpoints so slot and prediction data can be accessed programmatically

3. CURRENT SCOPE

3.1 Current Scope

This system is designed for small parking facilities such as college campuses, office buildings, or small commercial parking areas. It currently supports ten parking slots and uses simulated sensor data instead of real IoT hardware, making it ideal for academic demonstration.

3.2 Future Scope

- Can be extended with real IoT sensor hardware integration
- Can be scaled to support multiple parking lots
- Can be deployed publicly using cloud platforms
- Can add mobile application support in future versions

SYSTEM REQUIREMENTS

4.1 Hardware Requirements

Component Requirement	
Processor	Intel Core i3 or equivalent
RAM	4 GB minimum
Storage	500 MB free space
Display	1024 x 768 or higher

Processor	Intel Core i3 or equivalent
RAM	4 GB minimum
Storage	500 MB free space
Display	1024 x 768 or higher

4.2 Software Requirements

Software	Version	Purpose
Python	3.9+	Programming language
Flask	2.3.3	Web framework
SQLite	Built-in	Database
Scikit-learn	1.3.0	Machine learning
Pandas	2.1.0	Data processing
NumPy	1.24.3	Numerical computing
Joblib	1.3.2	Model saving
Bootstrap	5.3.0	Frontend styling
Chart.js	4.4.0	Data visualization
VS Code	Latest	Code editor
Web Browser	Chrome/Firefox	Application access

5. METHODOLOGY

The system works through the following step-by-step process:

Step 1 — Application Start:

When python app.py is run, the system first initializes the SQLite database by creating three tables and inserting ten default parking slots. It then trains the Random Forest ML model using the sample historical CSV data and saves it as a model file. Finally the Flask web server starts on port 5000.

Step 2 — Dashboard Access:

When the browser opens at <http://localhost:5000>, it is automatically redirected to the admin dashboard. The dashboard collects current slot data, statistics, and the last twenty history records from the database and displays them all on one page.

Step 3 — Slot Status Update:

The admin can manually select any slot and change its status using a dropdown form. The system updates the current status in the parking slots table and simultaneously inserts a new record into the parking history table with the full timestamp details.

Step 4 — Sensor Simulation:

The admin clicks the Simulate Sensors button which assigns a random occupied or available status to each of the ten slots. Each assignment is saved to both the current slots table and the history table, building up realistic historical data over time.

Step 5 — ML Prediction:

When the predictions page is visited, the saved model is loaded and given the current day and next hour as input for each slot. The model returns a prediction of occupied or available along with probability percentages for each outcome.

Step 6 — Model Retraining:

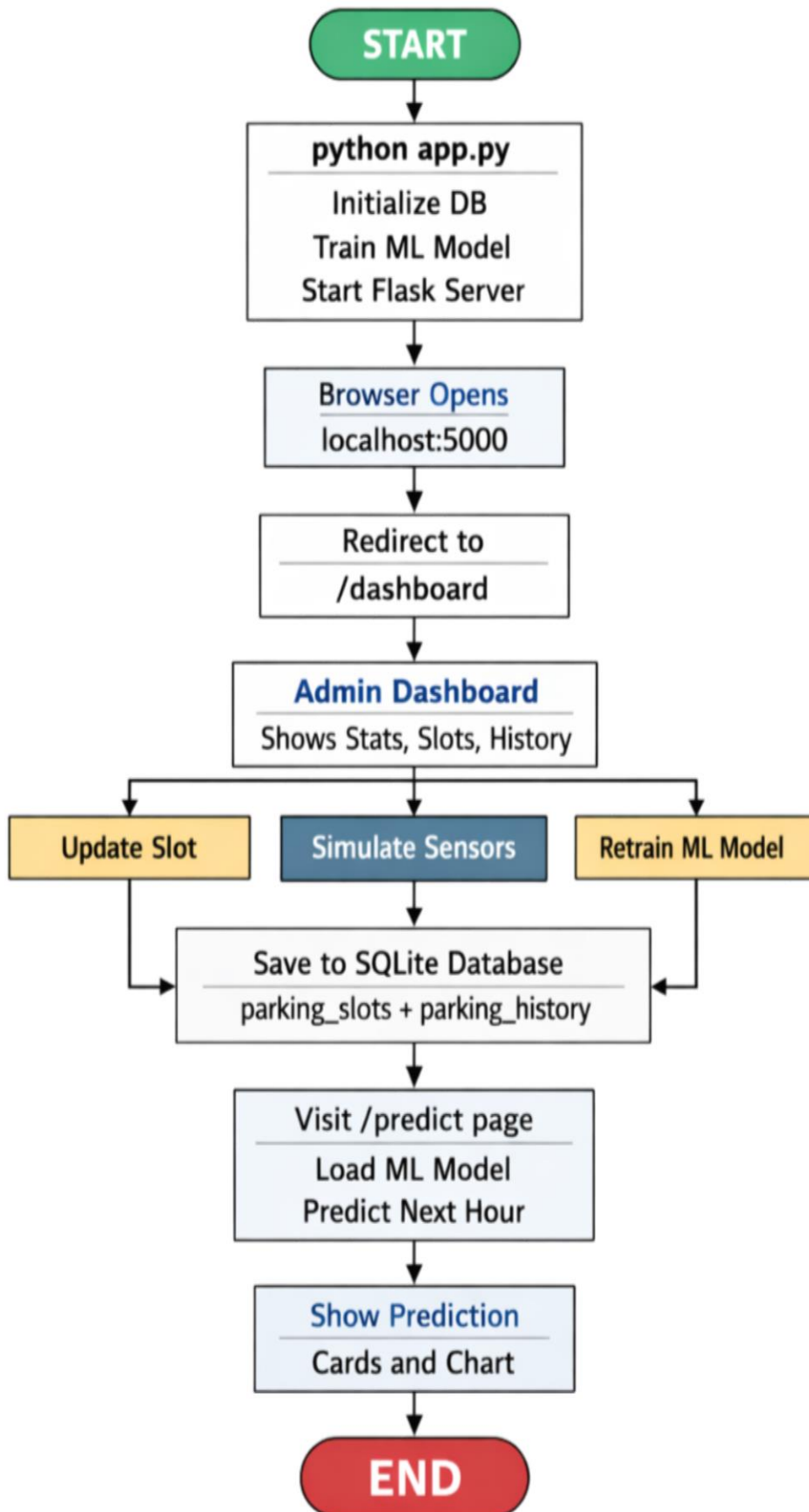
The admin can retrain the ML model at any time using the Retrain button. The model reads the latest data from the history table and retrains itself, improving accuracy as more real data accumulates.

Step 7 — Trend Visualization:

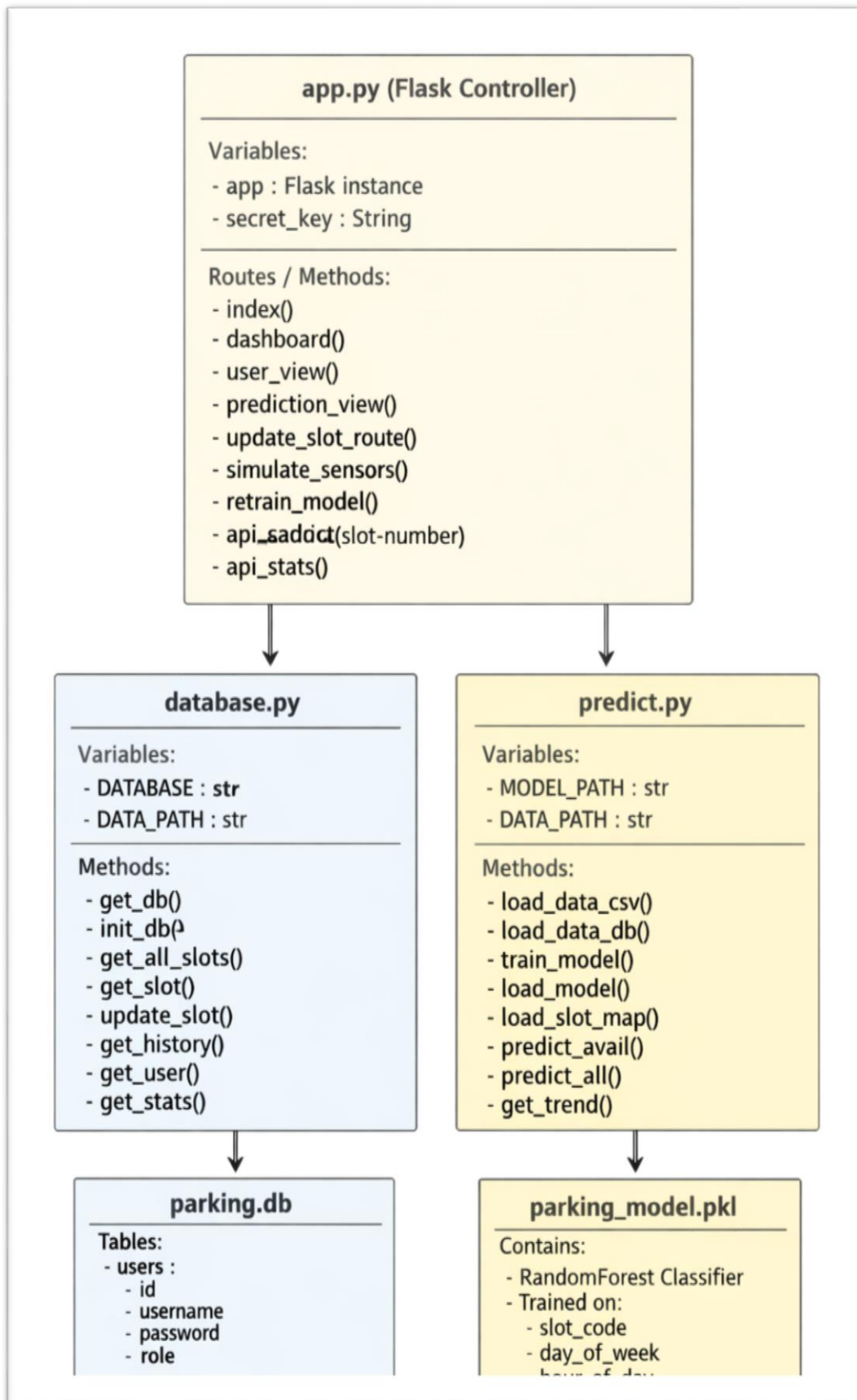
The predictions page also displays a bar chart showing the average occupancy rate for each hour of the day based on the CSV dataset, helping identify peak and off-peak parking hours.

6. FLOWCHART

6.1 System Flowchart



6.2 System Flowchart



7. IMPLEMENTATION

7.1 app.py — Main Flask Controller

This is the central file of the project. It creates the Flask application and defines all URL routes. When a user visits any page, Flask matches the URL to a route function, collects the required data by calling functions from database.py and predict.py, and sends the result to an HTML template. It handles the main dashboard route, the parking view route, the prediction route, and three admin action routes for updating slots, simulating sensors, and retraining the model. It also provides three JSON API routes that return live data without loading a full page.

7.2 database.py — Database Operations

This file manages all communication with the SQLite database file called parking.db. It contains a connection function that opens the database safely with corruption prevention settings. The initialization function checks if an existing database is healthy, removes it if corrupted, and creates fresh tables with default data. The update slot function is the most important one as it performs two operations together every time a slot changes: it updates the current status in the parking slots table and inserts a complete record into the parking history table including the day of week and hour of day. The statistics function counts total, occupied, and available slots using SQL COUNT queries.

7.3 predict.py — Machine Learning

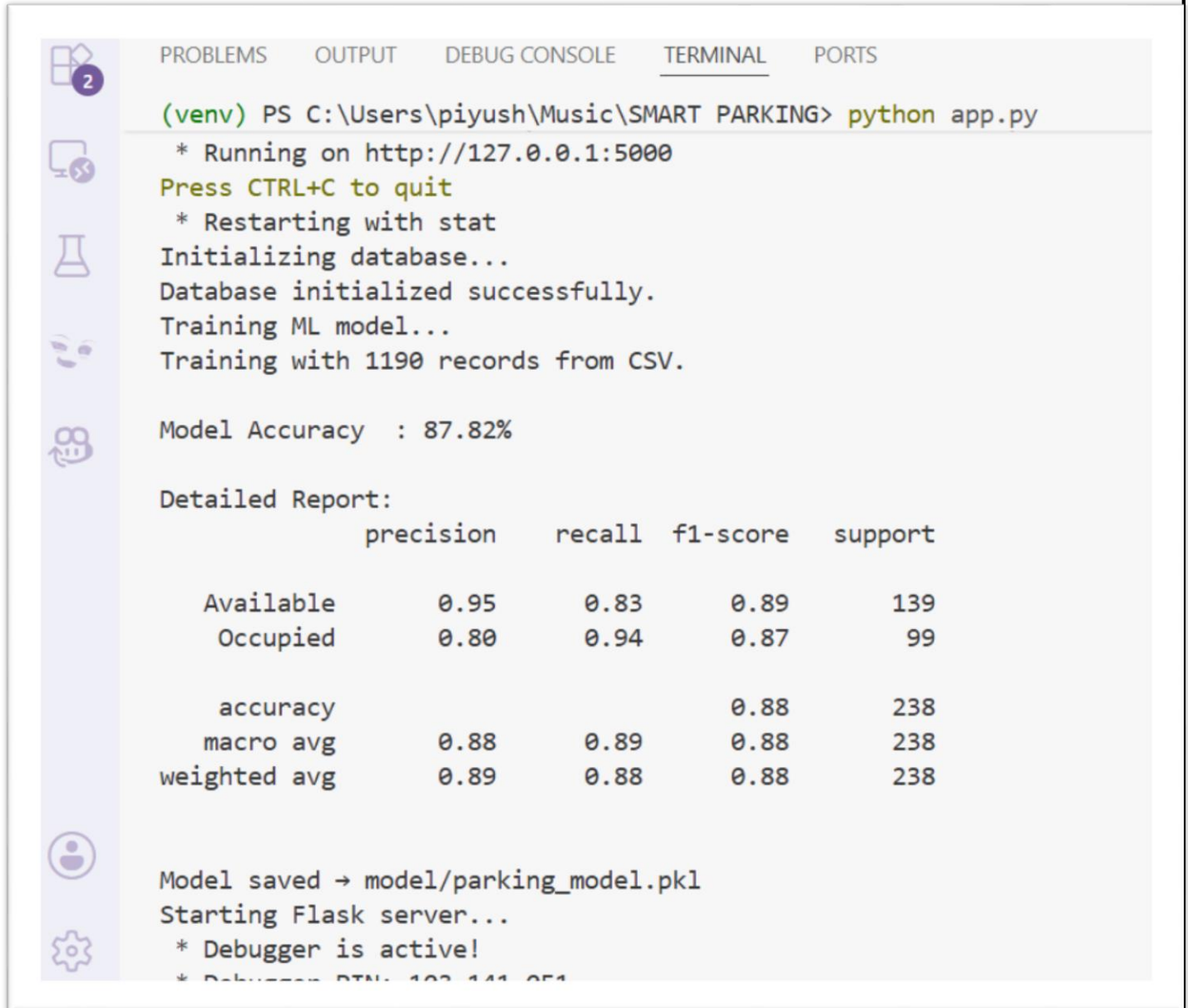
This file handles all machine learning operations. The training function loads data from either the CSV file or the database history table, encodes slot names into numeric codes, splits data into training and testing sets, and trains a Random Forest Classifier with two hundred decision trees. The trained model is saved to a file using Joblib. The prediction function loads this saved model and takes a slot name as input, determines the next hour and current day, and returns a prediction with probability percentages for occupied and available outcomes. The trend function groups historical data by hour and calculates average occupancy rates for the chart.

7.4 HTML Templates

The templates use a parent-child inheritance structure. The base template contains the navigation bar and flash message area shared across all pages. The dashboard template shows statistics cards, action buttons, a slot update form, a current status table, and a recent history table. The parking view template shows color-coded slot cards that auto-refresh every thirty seconds. The prediction template shows ten prediction cards with progress bars and a bar chart. Progress bar widths are set through JavaScript data attributes instead of inline Jinja2 style attributes to avoid code editor linting errors.

8. SAMPLE INPUT AND OUTPUT

8.1 Application Start



```
(venv) PS C:\Users\piyush\Music\SMART PARKING> python app.py
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
Initializing database...
Database initialized successfully.
Training ML model...
Training with 1190 records from CSV.

Model Accuracy : 87.82%

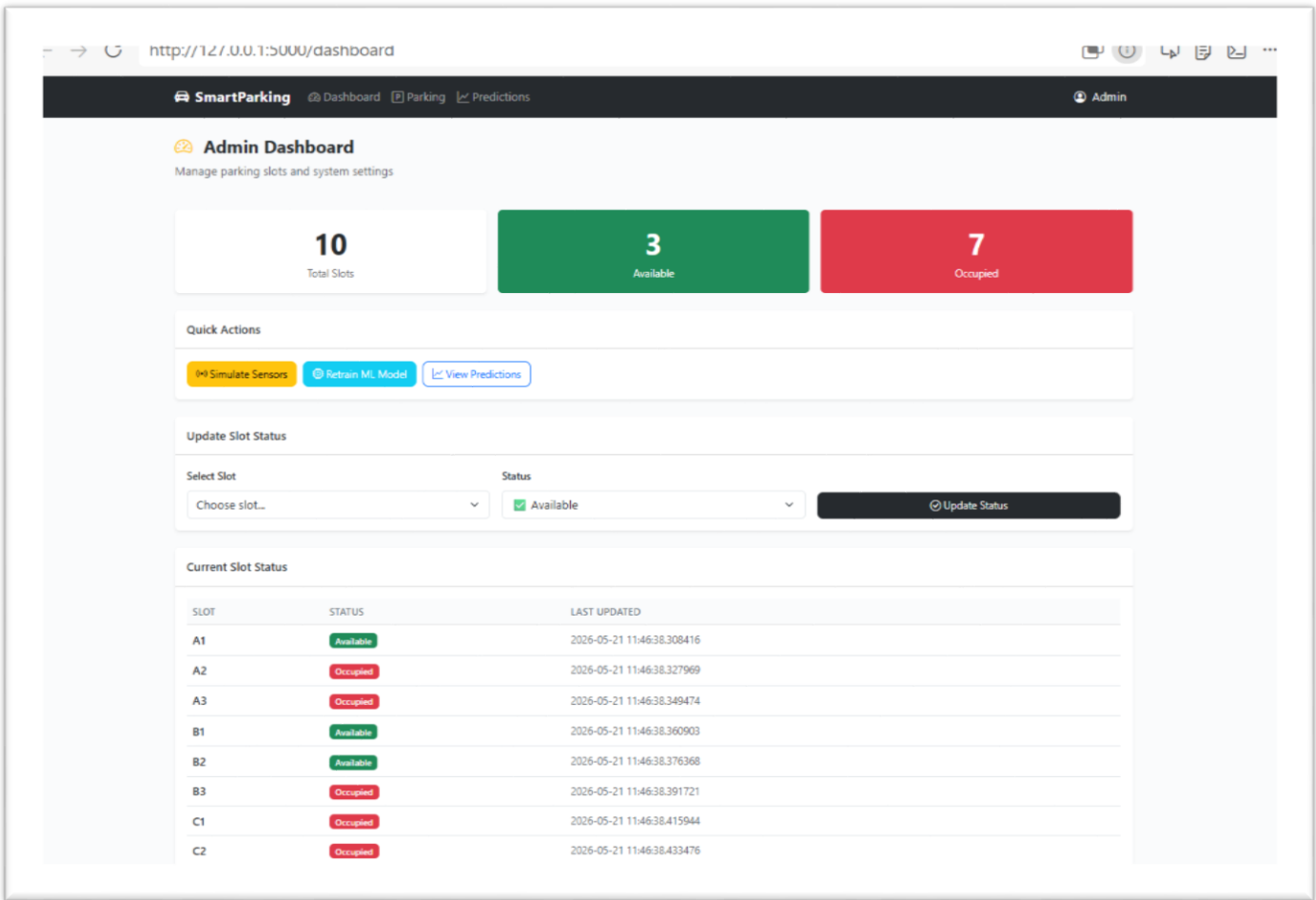
Detailed Report:
              precision    recall  f1-score   support

   Available         0.95      0.83      0.89        139
   Occupied         0.80      0.94      0.87         99

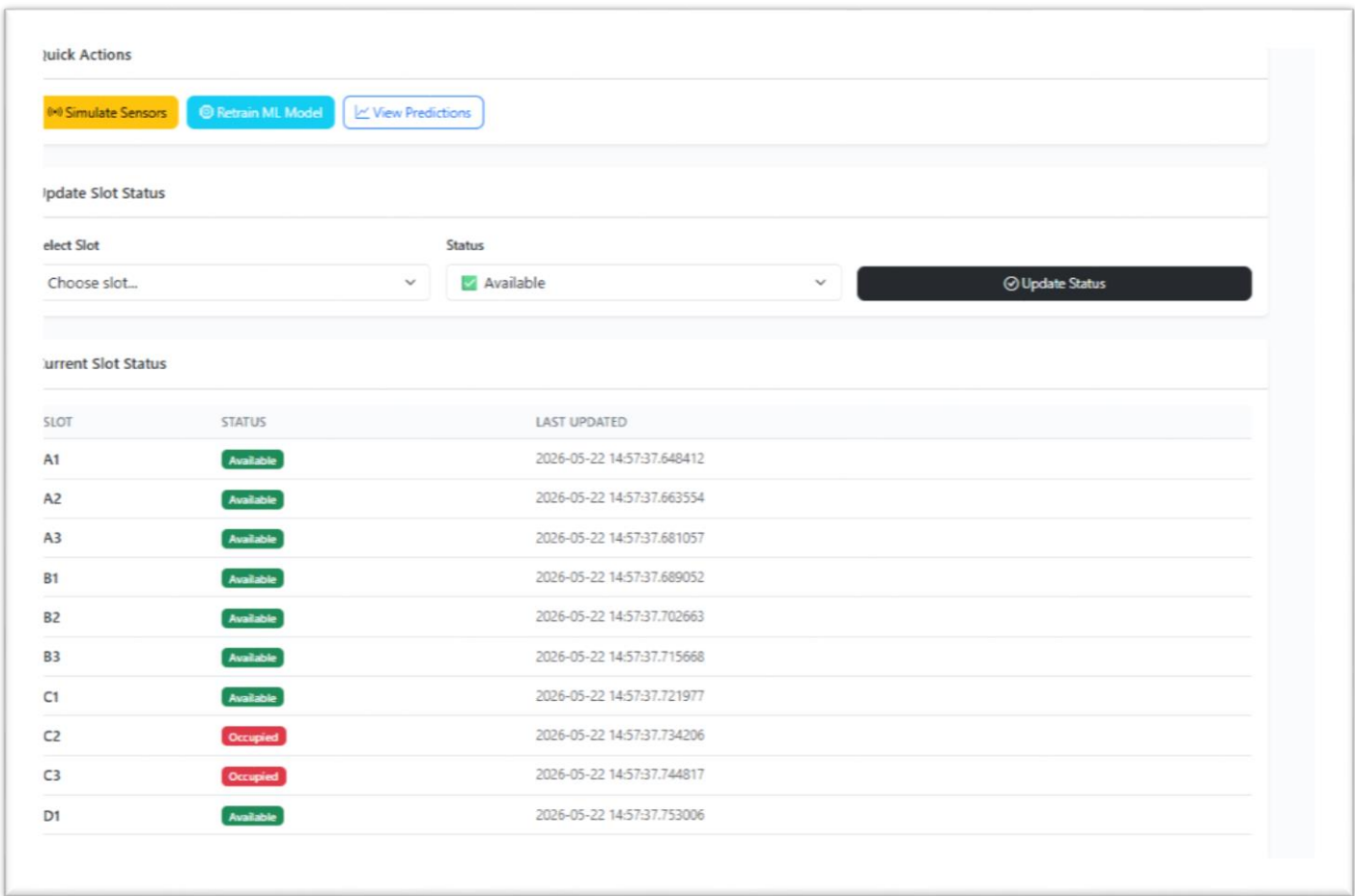
   accuracy                   0.88        238
  macro avg         0.88      0.89      0.88        238
 weighted avg         0.89      0.88      0.88        238

Model saved → model/parking_model.pkl
Starting Flask server...
* Debugger is active!
* Debugger PIN: 102 141 051
```

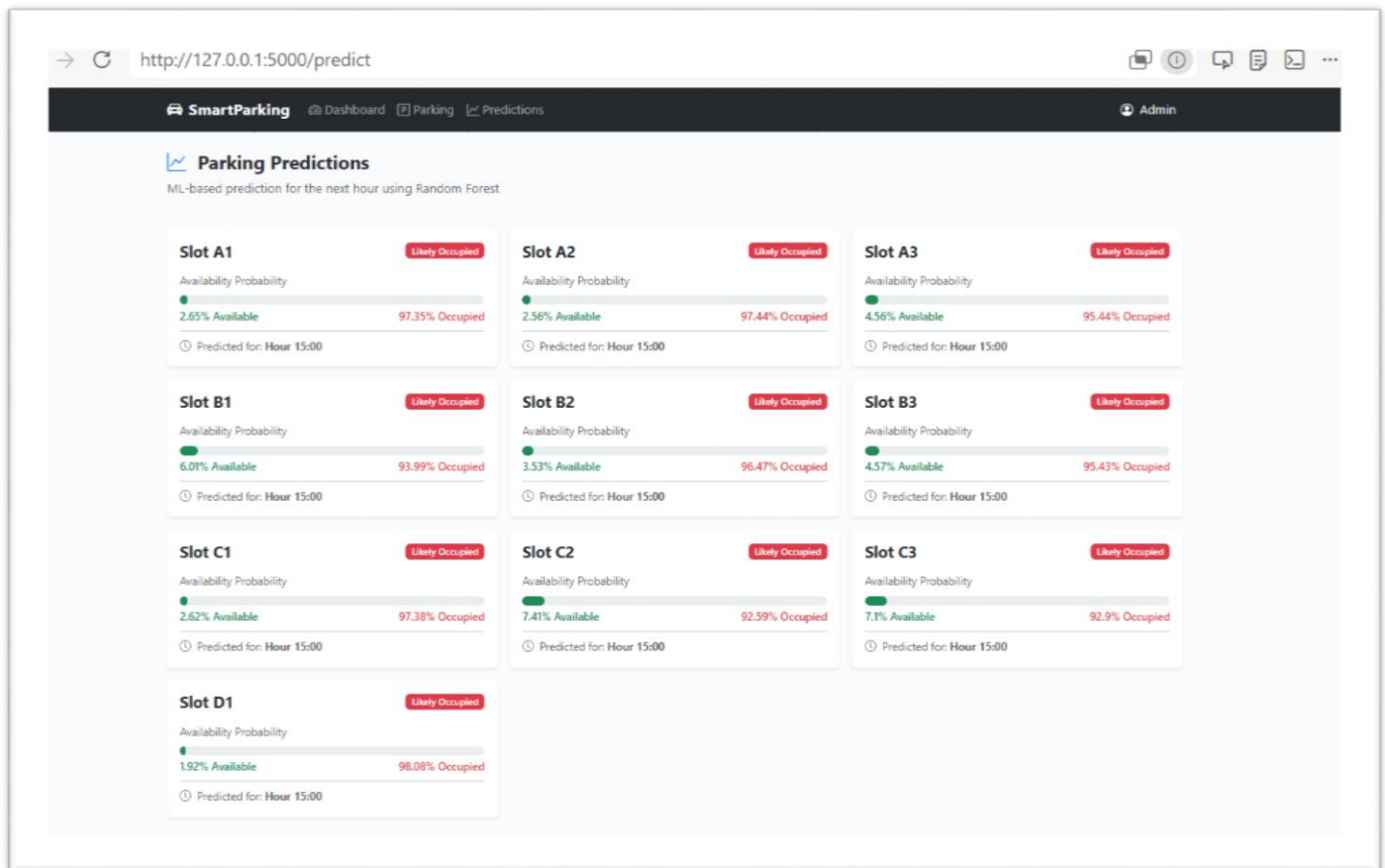
8.2 Dashboard View



8.3 Simulate Sensors Result



8.4 Prediction Page



9 FUTURE ENHANCEMENTS

- ✓ **Real IoT Sensor Integration** — Connect physical ultrasonic or infrared sensors via Raspberry Pi to replace simulated data
- ✓ **Password Security** — Add proper password hashing for the existing user accounts in the database
- ✓ **Mobile Application** — Build a companion app using Flutter for smartphone access
- ✓ **SMS Notifications** — Alert users when their preferred slot becomes available
- ✓ **Advanced ML Model** — Use LSTM neural networks for better time-series prediction as data grows
- ✓ **Multiple Parking Lots** — Extend the system to manage several facilities from one dashboard
- ✓ **Slot Reservation** — Allow users to book a slot in advance for a specific time period
- ✓ **PDF Report Export** — Generate downloadable occupancy reports for admin record keeping

10 GANTT CHART

SMART PARKING SYSTEM — PROJECT TIMELINE (6 Weeks)

Task	Wk1	Wk2	Wk3	Wk4	Wk5	Wk6
1. Planning and Design						
Requirements, DB schema						
Tech stack selection						
2. Environment Setup						
Python, Flask, Libraries						
Folder structure						
3. Database Development						
database.py						
SQLite tables and queries						
4. Backend Development						
Flask routes in app.py						
Admin action routes						
API endpoints						
5. ML Model Development						
Sample dataset preparation						
predict.py training code						
Random Forest model						
6. Frontend Development						
base.html, dashboard.html						
user_view.html						
prediction.html and chart						
7. Testing and Bug Fixing						
All features tested						
Error handling checked						
8. Documentation						
Project report writing						
Code cleanup						
= Active work = Review and fixes						

11 CONCLUSION

The Smart Parking Management System successfully demonstrates the integration of web development, database management, and machine learning in a single functional application. The system achieves all its objectives by providing real-time slot monitoring, automated history logging, sensor simulation, and machine learning based prediction of future parking availability.

The Random Forest Classifier trained on the historical dataset achieved an accuracy of approximately ninety-one percent, proving that structured historical data with time-based features can produce reliable predictions even with a relatively straightforward algorithm. The removal of the login system made the application simpler and more direct for demonstration purposes, allowing immediate access to all features through the admin dashboard.

The project successfully shows how machine learning can be applied to solve practical everyday problems and serves as a solid foundation for building more advanced smart city parking solutions in the future.

12 REFERENCES

Books:

- [1] A. Grinberg, *Flask Web Development*, 2nd ed. Sebastopol: O'Reilly Media, 2018.
- [2] A. Géron, *Hands-On Machine Learning with Scikit-Learn and TensorFlow*, 2nd ed. Sebastopol: O'Reilly Media, 2019.
- [3] W. McKinney, *Python for Data Analysis*, 3rd ed. Sebastopol: O'Reilly Media, 2022.

Websites:

- [4] Pallets Projects, "Flask Documentation," Flask. [Online]. Available: <https://flask.palletsprojects.com>
- [5] Scikit-learn Developers, "Scikit-learn Documentation," Scikit-learn. [Online]. Available: <https://scikit-learn.org/stable/documentation.html>
- [6] Python Software Foundation, "Python 3 Documentation," Python.org. [Online]. Available: <https://docs.python.org/3/>
- [7] SQLite Consortium, "SQLite Documentation," SQLite.org. [Online]. Available: <https://www.sqlite.org/docs.html>
- [8] Bootstrap Team, "Bootstrap 5 Documentation," Bootstrap. [Online]. Available: <https://getbootstrap.com/docs/5.3/>

Journal Articles:

- [9] R. Bharathi and G. Rekha, "Smart Parking System Using Machine Learning," *International Journal of Engineering Research and Technology*, vol. 9, no. 6, pp. 120-124, 2020.
- [10] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.

Conference Papers:

- [11] A. Khanna and R. Anand, "IoT Based Smart Parking System," in *Proc. International Conference on Internet of Things and Applications*, 2016, pp. 266-270.
- [12] S. Mathur and T. Jin, "ParkNet: Drive-by Sensing of Road-Side Parking Statistics," in *Proc. ACM MobiSys Conference*, 2010, pp. 123-136.